EE5203 L04 Practice Worksheet

Pointers, bits, masks, and CMSIS registers

Basri Erdogan

Time: 30-45 minutes. **Rule:** try every question before you open the solutions. Answer the register questions on paper first.

Part A - Pointers and structures (8 min)

1. Trace by hand and give every final value:

```c
/* (a) */                       /* (b) */
int a = 3;                      int m = 10;
int *q = &a;                    int n = 20;
                                int *p = &m;
*q = *q * 4;
a = a + 1;                      *p = *p + n;
                                p = &n;
                                *p = 0;
```

2. `struct Point { int x; int y; };` and `struct Point pt = {4, 7};` `struct Point *pp = &pt;` — for each expression, state its value or say it is invalid:

   a. `pt.x`
   b. `pp->y`
   c. `(*pp).x`
   d. `*pp.y`

3. Explain in one sentence why the device header defines `GPIOA` as a **pointer** to a structure rather than a structure variable.

Part B - Binary and hexadecimal (7 min)

4. Convert:

   a. `0x0C00` to binary (16 bits) — which bits are set?
   b. `0x00200000` to the single set bit's index.
   c. binary `0000 0100 0001 0001` to hex.
   d. `1U << 13` to hex.

5. GPIOA's `MODER` reads `0xA8000400`. Which pins are **not** in mode `00`, and what is the mode of PA5? (Pin `n` owns bits `2n+1:2n`.)

Part C - Masks (10 min)

6. Write one statement each (operate on `uint32_t value`):

   a. set bit 12;
   b. clear bit 3;
   c. toggle bit 0 (hint: `^` flips a bit);
   d. test whether bit 9 is set, as an `if` condition (hint: `value & mask` is non-zero when that bit is `1`);
   e. clear the two-bit field at bits 7:6, then write pattern `10` into it.

7. `value` starts as `0x0000 00F0`. Give the value after **each** statement, applied in order:

```
value |=  (1U << 8);
value &= ~(1U << 5);
value ^=  (1U << 4);
value &= ~(3U << 2);
```

8. A student wants PA3 (mode bits 7:6) as output and writes only `GPIOA->MODER |= (1U << 6);`
   a. For which starting field values does this produce the wrong mode?
   b. Write the correct two-statement sequence.
9. Compare `if (flags & (1U << 4))` and `if (flags && (1U << 4))`. Say in one sentence what each one tests. Then give one value of `flags` for which the two answers are different.

Part D - CMSIS registers and GPIO (10 min)

10. Decode `RCC->AHB1ENR |= (1U << 0);` word by word: `RCC`, `->`, `AHB1ENR`, `|=`, `1U << 0`. Why is `|=` essential here rather than `=`?
11. An LED is connected to PB0 and a push button to PC1. Write the statements to:
    a. enable the clocks for GPIOB **and** GPIOC in one statement;
    b. make PB0 an output (field bits 1:0);
    c. make PC1 an input with pull-up (`MODER` bits 3:2 = `00`, `PUPDR` bits 3:2 = `01`);
    d. test in an `if` whether PC1 currently reads low (button pressed), using `GPIOC->IDR`.
12. Why is plain assignment (`=`) correct for `GPIOA->BSRR` but destructive for `GPIOA->MODER`? Name the register property that makes the difference.
13. Write the BSRR statements that drive PA5 high and low, and state the general rule for the reset bit of pin `n`.

Part E - Predict the register value (5 min)

14. `GPIOA->MODER` reads `0xA8000400`, the value after the CubeMX initialisation. Then the two `MODER` statements of `led_init()` run:

```
GPIOA->MODER &= ~(3U << 10);   /* value now? */
GPIOA->MODER |=  (1U << 10);   /* value now? */
```

    Write the hex value of `MODER` after each statement.
15. In the SFR view you observe `RCC->AHB1ENR = 0x00000085`. Which GPIO ports are clocked? (AHB1ENR bit `n` clocks GPIO port `n`: A=0, B=1, C=2, … H=7.)

Part F - Check it in the debugger (5 min)

16. LD2 blinks. Show in the debugger that **your** two `MODER` statements make PA5 an output, not the CubeMX code. Write where you put the breakpoint and which `MODER` bits you read, before and after each statement.
17. The build is clean, but LD2 never lights. Give three different register mistakes that cause this. For each one, write the register and the bit value you would check in the register (SFR) view.

Exit self-assessment

Mark one:

- ☐ Ready for Lab 3 (6/7 October) without more review
- ☐ Need to review pointers and `->`
- ☐ Need to review binary/hex and shifts
- ☐ Need to review the mask recipes
- ☐ Need to review the RCC/MODER/BSRR roles